

Can we Speed up Information Set Decoding by Using Extension Field Structure?

Freja Elbro¹ and Violetta Weger²

¹Technical University of Denmark, Denmark

²Technical University of Munich, Germany

August 1, 2025

Abstract

The Syndrome Decoding Problem (SDP) underpins the security of most code-based cryptographic schemes, and Information Set Decoding (ISD) algorithms are the fastest known solvers for most parameter sets. While ISD is well developed in the binary setting, the landscape for non-binary ISD is less mature. Most q -ary methods are straightforward generalizations of their binary counterparts, with the recent projective Stern algorithm being the only exception. However, no existing algorithm is designed to leverage the specific algebraic properties of extension fields. This research gap – highlighted by the first-round NIST PQC proposal SDitH – motivates our central question: is decoding over an extension field fundamentally easier than over a prime field of similar size?

This work explores whether the algebraic structure of extension fields can accelerate ISD. We analyze several techniques for translating the SDP to the base field, including the expansion map, subfield subcodes, and the trace map. We also develop new BJMM variants that restrict base list vectors to “small” field elements, aiming to counter the performance loss of advanced ISD when q is large.

Contrary to our initial intuition, our results provide no evidence of an asymptotic speedup, suggesting that decoding over extension fields is not easier than over prime fields. Additionally, we make two contributions of independent interest: we show that a three-level BJMM algorithm gives a slight improvement over the two-level version for small fields, and we extend Meurer’s proof to show that the complexity of advanced ISD algorithms converges to Prange’s, even when parameters grow simultaneously.

1 Introduction

The Syndrome Decoding Problem (SDP) is a cornerstone of code-based cryptography, providing the security foundation for numerous post-quantum cryptographic schemes. Given a parity-check matrix $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$, a syndrome $\mathbf{s} \in \mathbb{F}_q^{n-k}$, and a weight w , the SDP asks to find a vector $\mathbf{e} \in \mathbb{F}_q^n$ of Hamming weight w such that $\mathbf{H}\mathbf{e} = \mathbf{s}$. For solving the SDP, two algorithm families dominate: statistical decoding [31, 16, 13, 14] and Information Set Decoding (ISD). While the latest statistical decoding methods [14] show significant promise for lower rates, ISD remains the most powerful approach for the higher-rate parameters typical of many cryptographic applications [29, 2, 4, 1].

The core idea of ISD dates back to a 1962 paper by Prange, who observed that one only needs to guess the correct values of an error vector on a so-called information set; the remaining entries can then be determined by linear algebra [34]. While elegant, this approach has seen significant evolution, particularly for the binary case, with improvements using meet-in-the-middle techniques, representation-based decoding, nearest-neighbor searches and sieving [35, 18, 27, 8, 28, 10, 24, 17].

Some of these improvements have been generalized to non-binary fields [32, 30], but their asymptotic advantage diminishes as the field size q increases. As shown in [30, Theorem 7.1.2], the benefit of these advanced algorithms over Prange’s original method tends to zero as q grows. This has led to a curious state of affairs where security analyses for modern q -ary schemes often rely on decades-old algorithms [29, 20].

This gap raises a fundamental question, which is the focus of our work: can the algebraic structure of a non-binary field be exploited to accelerate ISD? This question has remained largely unexplored. To the best of our knowledge, the recently proposed Projective Stern algorithm is the only algorithm to break this trend, successfully working in the projective space to reduce list sizes [12]. The success of this singular approach affirms that exploiting the field structure is a fruitful research direction. Our article addresses this same core question, focusing specifically on the distinction between extension and prime fields. We ask the question:

Is decoding over an extension field fundamentally easier than over a prime field of comparable size?

This is a question of high practical relevance, as the uncertainty has led cryptographic proposals like SDitH to specify parameter sets for both extension and prime fields ($q = 256 = 2^8$ and $q = 251$, respectively) [3].

Motivated by this open question, we propose several distinct strategies for exploiting the extension field structure. We present and analyze a range of new approaches, including:

- Translating the decoding problem to its base field using tools like the expansion map, subfield subcodes and the trace map.

- Developing novel BJMM variants in which the base list vectors are restricted to contain only “small” field elements. The motivation for this approach arises from the critical dependency of base list sizes on the field size q , which is a central element in the proof showing that advanced ISD algorithms converge to Prange’s algorithm as q grows [30, Theorem 7.1.2]. By limiting the growth of base list sizes with q , or ideally making them independent of q , we aim to circumvent this convergence and thus maintain efficiency advantages over Prange’s algorithm for large fields.

Contrary to our initial intuition stemming from related problems in the rank metric [21] and for rings [15], our findings indicate that these methods do not provide an asymptotic speedup, suggesting a negative answer to our main question. Alongside this main result, this work also provides two contributions of independent interest:

- we show, as a new result, that a three-level BJMM outperforms the standard two-level version [30]
- and we expand on the proof by Meurer [30, Theorem 7.1.2]. Meurer’s proof analyzed the asymptotic behavior by first letting the code length, n , go to infinity while the field size, q , remains fixed, and subsequently letting q go to infinity. Our work extends this by analyzing the more general case where n and q grow simultaneously.

After completing our research, we were made aware of Kevin Carrier’s thesis [11], which introduces an ISD algorithm that asymptotically outperforms BJMM and matches the speed of May–Ozerov in the binary setting. We were not aware of his work during our research phase, and consequently did not compare our results against his. However, since we do not achieve a runtime improvement over BJMM, incorporating Carrier’s findings would not alter our conclusions.

1.1 Organization

The remainder of this paper is organized as follows. We begin by reviewing essential preliminaries and surveying important Information Set Decoding algorithms in Sections 2 and 3, respectively. In Section 4, we then establish an asymptotic baseline by comparing these algorithms; our analysis reveals that a three-level BJMM offers a slight improvement over the standard two-level version for small fields.

The core of our work begins in Section 5, where we extend [30, Theorem 7.1.2] to show that the convergence holds even when parameters grow simultaneously. Following this, Section 6 details our main investigation: a series of attempts to exploit extension field algebra. These strategies include translating the problem to the base field (Sections 6.1 and 6.3) and novel BJMM variants using low-weight elements (Section 6.2). We conclude in Section 7 by summarizing our findings and outlining promising directions for future research.

2 Preliminaries

In this section we introduce all the relevant notation and background for the course of this paper. We also provide an overview of the existing ISD algorithms, which are oblivious to the underlying field structure.

2.1 Notation

We start by fixing the notation that is used throughout the paper. We denote by $[a; b]$ the set of integers between a and b . For a set $S \subseteq [1; n]$, we denote by $S^C = [1; n] \setminus S$ its complement and by $|S|$ its cardinality. Let us denote by $\mathbb{F}_q$ a finite field with q elements, where q is a prime power. We denote matrices, respectively vectors by bold upper case, respectively lower case letters, e.g. $\mathbf{A} = (a_{i,j})_{i \in [1;k], j \in [1;n]}$ is a $k \times n$ matrix. All vectors are considered as column vectors unless specified otherwise. The identity matrix of size n is denoted by $\mathbf{I}_n$, whereas the all zero matrix, or vector is denoted by $\mathbf{0}$ and its size will be clear from the context.

For a matrix $\mathbf{A} \in \mathbb{F}_q^{k \times n}$ and a set $S \subset [1; n]$ of size $|S| = s$, we denote by $\mathbf{A}_S$ the $k \times s$ matrix consisting of the columns of $\mathbf{A}$ indexed by S . We use $\mathcal{O}$ to denote the Landau symbol, which hides constant factors, and its variant $\tilde{\mathcal{O}}$, which also hides polylogarithmic factors. Formally, we define $f(n) \in \tilde{\mathcal{O}}(h(n))$ to mean that there exists a constant k such that $f(n) \in \mathcal{O}(h(n) \log^k(h(n)))$.

We use the multinomial coefficient $\binom{n}{k_1, \dots, k_t}$ to count the number of ways to partition a set of n elements into t distinct subsets of sizes $k_1, \dots, k_t$ that satisfy $\sum_{i=1}^t k_i = n$. It is given by the formula:

$$\binom{n}{k_1, \dots, k_t} = \frac{n!}{k_1! \cdots k_t!}$$

As $k_1, \dots, k_t$ must sum to n , the size of the last subset is determined by the others. We therefore use a dot as a convenient shorthand:

$$\binom{n}{k_1, \dots, k_{t-1}, \cdot} \triangleq \binom{n}{k_1, \dots, k_{t-1}, n - \sum_{i=1}^{t-1} k_i}$$

2.2 Entropy and Estimates of Binomial Coefficients

The q -ary entropy function $H_q : [0, 1] \rightarrow [0, 1]$ is defined as

$$H_q(x) = x \log_q(q-1) - x \log_q(x) - (1-x) \log_q(1-x),$$

with the convention that $0 \log_q 0 = 0$. The function is strictly increasing on the interval $[0, 1 - 1/q]$, and hence invertible on this domain.

The binary entropy function $H \triangleq H_2$ gives the well-known asymptotic estimate

$$\binom{n}{k} \in \tilde{\mathcal{O}}\left(2^{nH(k/n)}\right) \quad (1)$$

as $n \rightarrow \infty$, which can be proven using Stirling's approximation.

More generally, for fixed $q \geq 2$, we have

$$\binom{n}{w} (q-1)^w \in \tilde{\mathcal{O}}\left(q^{nH_q(w/n)}\right),$$

which we will use to estimate the number of weight- w vectors in $\mathbb{F}_q^n$.

2.3 Coding Theory

We briefly recall some of the main concepts from coding theory that are used throughout this work.

A *linear code* $\mathcal{C} \subseteq \mathbb{F}_q^n$ of dimension k is a k -dimensional subspace of the vector space $\mathbb{F}_q^n$. The elements of $\mathcal{C}$ are called *codewords*, and the *rate* of the code is defined as $R = k/n$.

A code can be specified either by a *generator matrix* $\mathbf{G} \in \mathbb{F}_q^{k \times n}$, whose rows span $\mathcal{C}$, or by a *parity-check matrix* $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$, which defines $\mathcal{C}$ as its kernel:

$$\mathcal{C} = \{\mathbf{c} \in \mathbb{F}_q^n \mid \mathbf{H}\mathbf{c} = \mathbf{0}\}.$$

Given a vector $\mathbf{x} \in \mathbb{F}_q^n$, its *syndrome* through $\mathbf{H}$ is defined as $\mathbf{s} = \mathbf{H}\mathbf{x}$.

A central notion in decoding is that of an *information set*. Given a linear code of dimension k , an information set is a subset $J \subset [1; n]$ of size k such that the columns of the generator matrix $\mathbf{G}$ indexed by J are linearly independent. Equivalently, the submatrix $\mathbf{G}_J$ is invertible. This ensures that the restriction of the code to the positions in J retains all the information of the original code.

The same condition can be expressed in terms of the parity-check matrix: J is an information set if the submatrix $\mathbf{H}_{J^C}$ is invertible, where $J^C = [1; n] \setminus J$.

The existence of an information set allows us to bring both $\mathbf{G}$ and $\mathbf{H}$ into *systematic form*. That is, there exists a permutation matrix $\mathbf{P}$ such that:

$$\mathbf{G}\mathbf{P} = (\mathbf{I}_k \quad \mathbf{A}), \quad \mathbf{H}\mathbf{P} = (\mathbf{B} \quad \mathbf{I}_{n-k}),$$

for some matrices $\mathbf{A} \in \mathbb{F}_q^{k \times (n-k)}$ and $\mathbf{B} \in \mathbb{F}_q^{(n-k) \times k}$.

We measure distances between vectors using the *Hamming metric*. The Hamming weight of a vector $\mathbf{x} \in \mathbb{F}_q^n$ is the number of non-zero entries:

$$\text{wt}(\mathbf{x}) = |\{i \in [1; n] \mid x_i \neq 0\}|,$$

and the Hamming distance between two vectors $\mathbf{x}, \mathbf{y}$ is defined as $d(\mathbf{x}, \mathbf{y}) = \text{wt}(\mathbf{x} - \mathbf{y})$.

The number of vectors in $\mathbb{F}_q^n$ of Hamming weight at most r —the size of the Hamming ball of radius r —is denoted by $B_q(n, r)$ and can be calculated as

$$B_q(n, r) = \sum_{i=0}^r \binom{n}{i} (q-1)^i.$$

A key parameter of a code is its *minimum distance*, defined as the smallest Hamming weight of any non-zero codeword. We denote this by d , and define the *relative minimum distance* as $\delta = d/n$.

A **random linear code** is one whose generator matrix $\mathbf{G}$ is chosen uniformly at random from the set of all $k \times n$ matrices over $\mathbb{F}_q$ with full rank. For such a code of dimension k and length n over $\mathbb{F}_q$, the minimum distance satisfies the *Gilbert–Varshamov bound* [22, 36] with high probability as n grows [6, 33]. That is,

$$H_q(\delta) = 1 - R,$$

2.4 Syndrome Decoding Problem

The *Syndrome Decoding Problem* (SDP) is a central problem in code-based cryptography. It was originally introduced for binary linear codes and later shown to be NP-hard over any finite field $\mathbb{F}_q$ [9, 5].

In this work, we focus on the computational version of the problem:

Problem 1. Let $\mathbf{H} \in \mathbb{F}_q^{n-k \times n}$, $\mathbf{s} \in \mathbb{F}_q^{n-k}$ and w be a positive integer. Find $\mathbf{e} \in \mathbb{F}_q^n$ of weight w such that $\mathbf{s} = \mathbf{H}\mathbf{e}$.

The SDP is equivalent to other well-known problems in coding theory, such as the general decoding problem (recovering the original message from a received word) and the low-weight codeword problem (finding a non-zero codeword of small Hamming weight). All these problems reduce to finding a sparse solution to a linear system over a finite field.

We typically assume that $\mathbf{H}$ is chosen uniformly at random and that $\mathbf{e}$ is also random subject to the weight constraint. In many cryptographic applications, including this paper, we further assume *half distance decoding*, meaning that $w = \lfloor \frac{d-1}{2} \rfloor$, where d is the minimum distance of the code. In this setting, the solution to the SDP is guaranteed to be unique.

The security of cryptosystems relying on the SDP hinges on the computational difficulty of finding this unique solution. For many parameter regimes, the most effective attacks are part of the Information Set Decoding (ISD) family of algorithms, first introduced by Prange [34].

3 Background on Information Set Decoding

In this section, we present some of the most important Information Set Decoding (ISD) algorithms, including those by Prange [34], Stern and Dumer [35, 18], MMT and BJMM [27, 8], as well as more recent approaches based on nearest-neighbor search [28, 10]. All of these algorithms were initially developed for binary codes and later generalized to the q -ary case in [32, 30, 25].

Since our main focus is on ISD over $\mathbb{F}_q$, we introduce each algorithm directly in the q -ary setting. We outline their core ideas and state their asymptotic runtime. For further details, we refer the interested reader to [30].

Most ISD algorithms follow a common high-level structure. To avoid repetition, we first describe this general framework before discussing how specific algorithms refine it.

The problem setting is as follows: given a matrix $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$, a syndrome vector $\mathbf{s} \in \mathbb{F}_q^{n-k}$, and a target weight w , the goal is to find an error vector $\mathbf{e} \in \mathbb{F}_q^n$ of weight $\text{wt}(\mathbf{e}) = w$ such that $\mathbf{H}\mathbf{e} = \mathbf{s}$.

The ISD framework proceeds in the following steps:

1. *Permutation step:* Choose a subset $J \subset [1; n]$ of size $k + \ell$ for some integer $\ell \in [0, n - k]$ to be optimized for runtime. For a random code, this subset contains an information set with a constant probability (lower bounded by $1/4$). The selection is repeated until a subset containing an information set is found, a process requiring only a few attempts on average. Once such a set J is identified, let $\mathbf{P}$ be a permutation matrix that maps its coordinates to the leftmost $k + \ell$ positions.
2. *Gaussian Elimination Step:* Compute $\mathbf{H}\mathbf{P}$ and bring it into quasi-systematic form with partial Gaussian elimination. Thus

$$\mathbf{U}\mathbf{H}\mathbf{P} = \begin{pmatrix} \mathbf{H}' & \mathbf{0} \\ \mathbf{H}'' & \mathbf{I}_{n-k-\ell} \end{pmatrix},$$

for some $\mathbf{H}' \in \mathbb{F}_q^{\ell \times (k+\ell)}$, $\mathbf{H}'' \in \mathbb{F}_q^{(n-k-\ell) \times (k+\ell)}$ and an invertible matrix $\mathbf{U} \in \mathbb{F}_q^{(n-k) \times (n-k)}$ responsible for the partial Gaussian Elimination.

Perform the same operations on the syndrome $\mathbf{s}$, i.e., compute $\mathbf{U}\mathbf{s} = \begin{pmatrix} \mathbf{s}' \\ \mathbf{s}'' \end{pmatrix}$ where $\mathbf{s}' \in \mathbb{F}_q^\ell$ and $\mathbf{s}'' \in \mathbb{F}_q^{n-k-\ell}$. The sought-after error vector $\mathbf{e}\mathbf{P}$ can now be split into $\mathbf{e}' \in \mathbb{F}_q^{k+\ell}$ and $\mathbf{e}'' \in \mathbb{F}_q^{n-k-\ell}$ which fulfill

$$(\mathbf{U}\mathbf{H}\mathbf{P})(\mathbf{e}\mathbf{P}) = \begin{pmatrix} \mathbf{H}' & \mathbf{0} \\ \mathbf{H}'' & \mathbf{I}_{n-k-\ell} \end{pmatrix} \begin{pmatrix} \mathbf{e}' \\ \mathbf{e}'' \end{pmatrix} = \mathbf{U}\mathbf{s} = \begin{pmatrix} \mathbf{s}' \\ \mathbf{s}'' \end{pmatrix}.$$

Thus, the original problem is equivalent to

$$\begin{cases} \mathbf{H}'\mathbf{e}' = \mathbf{s}', \\ \mathbf{H}''\mathbf{e}' + \mathbf{e}'' = \mathbf{s}''. \end{cases}$$

and the error vector $\mathbf{e}$ can be calculated from $\mathbf{e}'$ and $\mathbf{e}''$ as

$$\mathbf{e} = \begin{pmatrix} \mathbf{e}' \\ \mathbf{e}'' \end{pmatrix} \mathbf{P}^{-1}.$$

3. *Solve the smaller instance:* Solve the smaller instance given by $\mathbf{H}'$, $\mathbf{s}'$ and target weight $w' \min\{w, k + \ell\}$, getting a list $\mathcal{L}$ containing candidates for $\mathbf{e}'$. w' is again a parameter of the algorithm chosen later to minimize runtime.
4. *Test step:* For all $\mathbf{e}' \in \mathcal{L}$, compute $\mathbf{e}'' = \mathbf{s}'' - \mathbf{e}'\mathbf{H}''$. If $\text{wt}(\mathbf{e}'') = w - w'$ return the solution $\mathbf{e} = \begin{pmatrix} \mathbf{e}' \\ \mathbf{e}'' \end{pmatrix} \mathbf{P}^{-1}$. Else, return to step 1 and choose a different set J .

To calculate the runtime of the ISD algorithms, we note that they consist of several iterations of one round (step 1 to step 4). We will write their cost as

$$T_{\text{ISD}} = \pi_{\text{ISD}} R_{\text{ISD}},$$

where π_{ISD} denotes the average number of iterations needed, and R_{ISD} denotes the cost of a round.

A chosen J will succeed if $\mathbf{e}$ has weight w' in the entries indexed by J , which happens for a random subset $J \subseteq [1; n]$ with probability

$$\binom{k+\ell}{w'} \binom{n-k-\ell}{w-w'} \binom{n}{w}^{-1}.$$

Thus

$$\pi_{\text{ISD}} = \binom{k+\ell}{w'}^{-1} \binom{n-k-\ell}{w-w'}^{-1} \binom{n}{w}. \quad (2)$$

The cost of a round is dominated by step 3, the solution to the smaller instance, and the partial Gaussian elimination (step 2). The first depends on the used technique in the different algorithms and the latter is estimated as

$$T_{\text{Gauss}} \in \mathcal{O}(n^3 \log(q)).$$

3.1 Prange

Prange's algorithm [34] from 1962 is the first ISD algorithm, and follows the described framework by setting $w' = \ell = 0$. Thus, $\mathbf{e}' = \mathbf{0}$ and finding this comes without cost. The runtime of **Prange** is thus

$$T_{\text{Prange}} \in \mathcal{O}\left(\binom{n}{w} \binom{n-k}{w}^{-1} n^3 \log(q)\right). \quad (3)$$

3.2 Stern and Dumer

In Stern's [35] and Dumer's [18] algorithms we solve the smaller instance by performing a meet-in-the-middle / collision search. Stern was the first to suggest this idea in 1989 and Dumer found a small improvement to it two years later. For the sake of brevity, we describe here only Dumer's algorithm.

Dumer writes $\mathbf{e}' = \begin{pmatrix} \mathbf{e}_1 \\ \mathbf{e}_2 \end{pmatrix}$ where $\mathbf{e}_i \in \mathbb{F}_q^{(k+\ell)/2}$ have weight $w'/2$. Thus, he also splits $\mathbf{H}'$ into $\mathbf{H}' = [\mathbf{H}_1 \ \mathbf{H}_2]$, where $\mathbf{H}_1, \mathbf{H}_2 \in \mathbb{F}_q^{\ell \times (k+\ell)/2}$ and construct two lists

$$\mathcal{L}_1 = \{(\mathbf{e}_1, \mathbf{H}_1 \mathbf{e}_1) \mid \mathbf{e}_1 \in \mathbb{F}_q^{(k+\ell)/2}, \text{wt}(\mathbf{e}_1) = w'/2\}$$

and

$$\mathcal{L}_2 = \{(\mathbf{e}_2, \mathbf{s}' - \mathbf{H}_2 \mathbf{e}_2) \mid \mathbf{e}_2 \in \mathbb{F}_q^{(k+\ell)/2}, \text{wt}(\mathbf{e}_2) = w'/2\}.$$

Thus, if we find a collision, i.e., $((e_1, \mathbf{a}), (e_2, \mathbf{a})) \in \mathcal{L}_1 \times \mathcal{L}_2$ we have a candidate for $e' = \begin{pmatrix} e_1 \\ e_2 \end{pmatrix}$, as $\mathbf{H}_1 e_1 = s' - \mathbf{H}_2 e_2$ implies that $\mathbf{H}' e' = s'$.

The cost of solving the smaller instance for Dumer's algorithm consists of constructing the lists $\mathcal{L}_i$ of size

$$B = \binom{(k+\ell)/2}{w'/2} (q-1)^{w'/2}$$

and the collision search. The average number of collisions, and thus the average size of $\mathcal{L}$ is given by

$$L = \frac{B^2}{q^\ell}.$$

Using a hash table for the collision search, we find that

$$R_{\text{Dumer}} \in \tilde{\mathcal{O}}(T_{\text{Gauss}} + B + L).$$

In Dumer's algorithm, the weight w' of the first $k + \ell$ coordinates of the permuted error vector e' must be evenly distributed between its two halves, e_1 and e_2 .¹ Consequently, the success probability in a single round becomes

$$\frac{\binom{(k+\ell)/2}{w'/2}^2 \binom{n-k-\ell}{w-w'}}{\binom{n}{w}}.$$

However, using the estimate from equation (1), it follows that this does not change the expected number of rounds π_{Dumer} by more than polynomial factors compared to the general form given in Equation (2).

3.3 MMT and BJMM

A more sophisticated idea is to use the representation technique introduced for subset sums in [26]. The idea was first used for decoding over the binary in [27] and has since been elaborated on in [8] and extended to arbitrary finite fields in [30].

The idea is to write $e' = e_1 + e_2$ with $e_1, e_2 \in \mathbb{F}_q^{k+\ell}$ both of weight $w_1 = s + o + d$. The two vectors e_1 and e_2 overlap in $o + d$ many positions, in such a way, that when adding them together, precisely d positions cancel out. We hence enlarge the size of the base lists. Additionally, we get many representations (e_1, e_2) leading to the same $e' = e_1 + e_2$.

In [30], the number of representations is computed as

$$\rho(q, k, \ell, w', s, o, d) \in \tilde{\mathcal{O}} \left(\binom{w'}{s, s, o} (q-2)^o \binom{k+\ell-w'}{d} (q-1)^d \right).$$

¹We assume that the optimal w' is even.

That is, if we go through all $\mathbf{e}_1, \mathbf{e}_2$ and add $\mathbf{e}' = \mathbf{e}_1 + \mathbf{e}_2$ to the list $\mathcal{L}$, then every candidate $\mathbf{e}' \in \mathbb{F}_q^{k+\ell}$ of weight w' is represented $\rho(q, k, \ell, w', \varepsilon, \delta, d)$ many times in that list. We, thus, do not have to go through all possible $\mathbf{e}_1, \mathbf{e}_2$. By requiring the vectors $\mathbf{e}_1, \mathbf{e}_2$ to add up to the desired partial syndrome on $0 \leq \ell_1 \leq \ell$ many positions, we can set $\ell_1 = \log_q(\rho(q, k, \ell, w', s, o, d))$ and achieve that one representation of $\mathbf{e}'$ is constructed on average. For this we need the following notation: we write $\mathbf{a} \equiv_{\ell_1} \mathbf{b}$ if $\mathbf{a}$ is equal to $\mathbf{b}$ in the last ℓ_1 positions.

The algorithm in [30] uses two levels, that is, we split the vectors $\mathbf{e}_1, \mathbf{e}_2$ even further, as $\mathbf{e}_1 = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{y}_1 \end{pmatrix}$ and $\mathbf{e}_2 = \begin{pmatrix} \mathbf{x}_2 \\ \mathbf{y}_2 \end{pmatrix}$, where $\mathbf{x}_i, \mathbf{y}_i \in \mathbb{F}_q^{(k+\ell)/2}$ for $i \in \{1, 2\}$ and all have weight $w_1/2$. That is, we use a splitting like in Stern/Dumer's algorithm and again split the parity-check matrix $\mathbf{H}' = [\mathbf{H}_1 \ \mathbf{H}_2]$.

To sum up, the steps of the algorithm are as follows

1. Build the base list

$$\mathcal{B} = \{\mathbf{x} \in \mathbb{F}_q^{(k+\ell)/2} \mid \text{wt}(\mathbf{x}) = w_1/2\}.$$

2. For all $(\mathbf{x}, \mathbf{y}) \in \mathcal{B}^2$, add $\mathbf{e}_1 = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix}$ to the list $\mathcal{L}_1$, if $\mathbf{H}_1 \mathbf{x} + \mathbf{H}_2 \mathbf{y} \equiv_{\ell_1} \mathbf{0}$.
3. For all $(\mathbf{x}, \mathbf{y}) \in \mathcal{B}^2$, add $\mathbf{e}_2 = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \end{pmatrix}$ to the list $\mathcal{L}_2$, if $\mathbf{H}_1 \mathbf{x} + \mathbf{H}_2 \mathbf{y} \equiv_{\ell_1} \mathbf{s}'$.
4. For all $(\mathbf{e}_1, \mathbf{e}_2) \in \mathcal{L}_1 \times \mathcal{L}_2$, add $\mathbf{e}' = \mathbf{e}_1 + \mathbf{e}_2$ to the list $\mathcal{L}$, if $\text{wt}(\mathbf{e}') = w'$ and $\mathbf{H}' \mathbf{e}' = \mathbf{s}'$.

Similarly to Dumer's algorithm, we can calculate the size of the base list $\mathcal{B}$ as

$$B = \binom{(k+\ell)/2}{w_1/2} (q-1)^{w_1/2}$$

and the expected size of $\mathcal{L}_1$ and $\mathcal{L}_2$ as

$$L_1 = B^2 q^{-\ell_1}.$$

Finally, when merging $\mathcal{L}_1$ and $\mathcal{L}_2$ to get $\mathcal{L}$ we find in expectation

$$C = L_1^2 q^{\ell-\ell_1}$$

collisions on all ℓ coordinates, because we already know that the first ℓ_1 indices collide.

Using a hash-table to find collisions, we get a cost per round of

$$R_{\text{BJMM}} \in \tilde{O}(T_{\text{Gauss}} + B + L_1 + C).$$

Similarly to Dumer's algorithm, we only construct vectors $\mathbf{e}_1$ and $\mathbf{e}_2$ that have the same weight on the first and second half of the coordinates. However, similarly to Dumer's algorithm, we argue that these make up a polynomial fraction of all the relevant vectors, and that the expected number of rounds is thus still given by equation 2 up to polynomial factors.

3.4 ISD Algorithms Leveraging Nearest-Neighbor Techniques

In the binary setting, the asymptotically fastest ISD algorithms incorporate nearest-neighbor search methods to accelerate the merging steps [28, 10]. Rather than requiring an exact match between vectors, these methods allow for a small Hamming distance, which better reflects the structure of the decoding problem and leads to improved asymptotic complexity. The May–Ozerov algorithm [28] achieves the best known asymptotic performance for this task.

In 2016, Hirose extended the May–Ozerov approach to general finite fields and applied it to the merging step in Stern’s ISD, but found no asymptotic improvement for $q \geq 3$ [25]. A subsequent work by Gueye, Klamti, and Hirose did claim an asymptotic gain by combining the generalized algorithm with BJMM [23]; however, we identified a critical error in their analysis.² In light of this error and Hirose’s original findings, we consider Meurer’s generalization of BJMM [30] the prevailing asymptotic baseline for ISD over non-binary fields.

4 Best Algorithm Asymptotically

Having identified the state-of-the-art for ISD algorithms over $\mathbb{F}_q$, we now perform a formal asymptotic analysis to compare their runtimes. Our aim is to establish a baseline for our proposed improvements. The analysis is conducted in the regime where the field size q is fixed and the code parameters scale with n according to

$$\lim_{n \rightarrow \infty} \frac{k(n)}{n} = R \quad \text{and} \quad \lim_{n \rightarrow \infty} \frac{w(n)}{n} = W,$$

where $R \in [0, 1]$ is the code rate. We fix the relative weight to $W := \frac{1}{2}H_q^{-1}(1 - R)$, which corresponds to half-distance decoding at the Gilbert–Varshamov bound.

The runtime of the ISD algorithms in this setting has the form

$$T_{\text{ISD}}(q, n, Rn, Wn) = 2^{C(q, R)n + o(n)},$$

where we denote by $C(q, R)$ the *complexity coefficient* of the given ISD algorithm. To facilitate a meaningful comparison, we also define the *maximal asymptotic complexity coefficient*:

$$C_{\text{ISD}}^*(q) := \max_{R \in [0, 1]} C(q, R).$$

This maximal coefficient provides a single, scalar metric for comparing algorithms for a given field size. Although an algorithm with a larger $C_{\text{ISD}}^*(q)$ could hypothetically be faster for specific rates R , empirical evidence suggests that the algorithm with the minimal coefficient is generally superior across all relevant

²Using their proposed optimal parameters, the calculated size of the base list exceeds their reported total runtime. We contacted the authors about this issue but received no response.

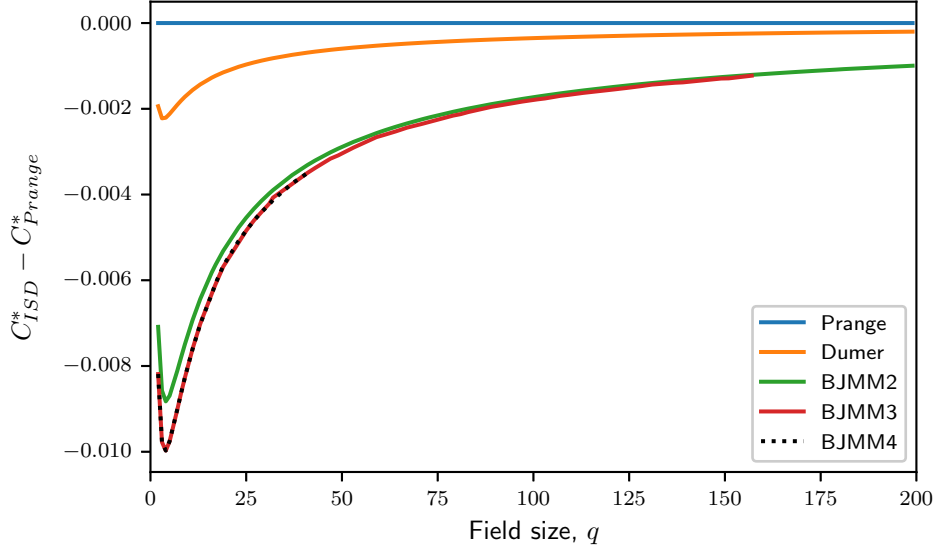


Figure 1: Difference between the maximal asymptotic complexity coefficient of Prange’s algorithm and those of various advanced ISD algorithms, as a function of the field size q .

parameters. This observation justifies using $C_{\text{ISD}}^*(q)$ as our primary metric for comparison.

To compute these coefficients, we used custom code³ that relies on the approximation from Eq. (1) for binomial coefficients. The code assumes that optimal internal parameters for each algorithm scale linearly with n and uses SciPy’s optimization library to find them. To mitigate the risk of settling in local minima, we performed hundreds of optimization runs with randomized starting points.

Our results, shown in Figure 1, lead to several conclusions. First, we confirm prior work [30] showing that two-level BJMM outperforms Dumer’s algorithm. We additionally find that three-level BJMM offers a slight improvement for small field sizes, while four levels provide no further benefit. Finally, our analysis confirms that the advantage of these advanced algorithms diminishes as the field size q grows. This aligns with [30, Theorem 7.1.2], which implies that

$$\lim_{q \rightarrow \infty} (C_{\text{Prange}}^*(q) - C_{\text{Stern}}^*(q)) = 0,$$

a result that the authors state also extends to BJMM.

The case for Prange’s algorithm is further strengthened when considering memory requirements. As noted in [30], when memory is highly constrained

³The code is available at github.com/FrejaElbro/ISD-for-extension-fields

(e.g., to 2^{40} operations), Prange’s memory-free algorithm becomes the most practical choice.

5 Asymptotic Analysis for Simultaneously Growing n and q

The previous section evaluated ISD algorithms using the coefficient $C^*(q)$, which is the standard approach for a fixed field size q . However, to analyze performance as q grows, prior work such as [30] has considered the double limit

$$\lim_{q \rightarrow \infty} \left(\lim_{n \rightarrow \infty} \frac{1}{n} \log_q (T_{\text{ISD}}(q, n, k, w)) \right).$$

This construct, which treats the limits sequentially, can be difficult to justify for practical scenarios where security parameters n and q grow together or where $q > n$. In this section, we address this by providing a novel asymptotic analysis for the case where q is a function of n .

First, we introduce notation to compare runtimes up to subexponential factors. Let $T_1(n)$ and $T_2(n)$ be two runtimes.

- We say the runtimes are **exponentially equivalent**, denoted $T_1 \overset{\text{exp}}{\approx} T_2$, if their ratio is subexponential, i.e., $T_1(n)/T_2(n) = 2^{o(n)}$.
- We write $T_1 \overset{\text{exp}}{\lesssim} T_2$ if T_1 is at most as large as T_2 in the exponential sense. Formally, this means $T_1 \overset{\text{exp}}{\approx} T_2$ or $\lim_{n \rightarrow \infty} \frac{T_1(n)}{T_2(n)} = 0$.

Theorem 2. *Let ISD be any algorithm covered in Section 3 and T_{ISD} denote its runtime with optimized internal parameters. Assume that k, w and q are all functions of n , which satisfy*

$$\lim_{n \rightarrow \infty} \frac{k(n)}{n} = R, \lim_{n \rightarrow \infty} \frac{w(n)}{n} = W \text{ and } \lim_{n \rightarrow \infty} q(n) = \infty,$$

for some $R, W \in [0, 1]$ with $W < 1 - R$ and $R > 0$. Then

$$T_{\text{ISD}} \overset{\text{exp}}{\approx} T_{\text{Prange}}.$$

Theorem 2 extends the result of Meurer [30] in two main regards:

1. It applies not just to Stern’s algorithm but to a general class of ISD algorithms, including BJMM, May-Ozerov [28], and Both-May [10] (the latter two with minor tweaks). Furthermore, the framework may cover future algorithms that follow the framework of [19].
2. It analyzes a more general asymptotic setting, particularly covering scenarios where $q > n$, which were not addressed in [30].

Proof. We begin by noting that, in the asymptotic regime of Theorem 2, the SDP has a unique solution with high probability for sufficiently large n . This follows from the condition $W < 1 - R$ and the uniform convergence of H_q^{-1} to the identity function.⁴ Hence the runtime formulas from Section 3 apply.

Next we note that all ISD algorithms contain Prange as a special case (e.g., via $w' = \ell = 0$), and hence we immediately have

$$T_{\text{ISD}} \stackrel{\text{exp}}{\lesssim} T_{\text{Prange}}.$$

It remains to show the reverse inequality.

As usual, we write the runtime as the expected number of rounds times the amount of work done per round

$$T_{\text{ISD}} = \pi_{\text{ISD}} \cdot R_{\text{ISD}}.$$

We then note that the only factor in R_{Prange} which can possibly grow exponentially is the one responsible for field arithmetic (and that factor is $O(\log(q))$). As field arithmetic is also required in the round function of ISD we must trivially have

$$R_{\text{Prange}} \stackrel{\text{exp}}{\lesssim} R_{\text{ISD}}.$$

To show the same holds for the expected number of rounds, observe that equation (3) implies

$$\log_2(T_{\text{Prange}}) \in O(\max(n, \log \log q))$$

and as $T_{\text{ISD}} \stackrel{\text{exp}}{\lesssim} T_{\text{Prange}}$, we must also have

$$\log_2(T_{\text{ISD}}) \in O(\max(n, \log \log q)). \quad (4)$$

The theorem is vacuously true for $\text{ISD} = \text{Prange}$, so from now on, let ISD be one of the other algorithms of Section 3. In ISD, the solution to the subproblem, $\mathbf{e}'$, is constructed as a sum of vectors with smaller Hamming weight. Let us denote by w_0 the weight of the vectors in the base list of the algorithm ISD. As all vectors $\mathbf{v} \in \mathbb{F}_q^{(k+\ell)/2}$ of weight w_0 are enumerated in the base lists of algorithm ISD, we must have

$$T_{\text{ISD}} \geq (q-1)^{w_0}.$$

From Equation (4) and $\lim_{n \rightarrow \infty} q = \infty$, it now follows that $w_0 \in o(n)$ and as $\mathbf{e}'$ is formed as a finite sum of vectors of weight w_0 , we must have that also

$$\text{wt}(\mathbf{e}') = w' \in o(n).$$

From this the claim follows as

$$\pi_{\text{ISD}} = \frac{\binom{n}{w}}{\binom{n-k-\ell}{w-w'} \binom{k+\ell}{w'}} \stackrel{\text{exp}}{\approx} \frac{\binom{n}{w}}{\binom{n-k-\ell}{w}} \geq \frac{\binom{n}{w}}{\binom{n-k}{w}} = \pi_{\text{Prange}}.$$

□

⁴which can be proven using [7, Theorem 1]

6 Attempts at Improving Information Set Decoding

Here we present an array of ideas for ISD algorithms over extension fields, with the aim of improving upon BJMM's asymptotic runtime. While none of the approaches achieved the goal, we would like to share our ideas and the valuable lessons we have learned through these attempts, to set this new line of research into motion: how can we exploit the extension field structure? At the end of this section we provide several promising ideas for further research and hope this survey will be helpful in resolving this long standing open question.

This section covers several attempts and requires a small guide. A core idea within this section is to make use of the expansion map $\Gamma : \mathbb{F}_{p^m} \rightarrow \mathbb{F}_p^m$. In Section 6.1 we will explore the idea of solving the original instance in the base field using this expansion. We find that this leads to a ISD instance over $\mathbb{F}_p$ with length mn dimension mk and a priori target weight mw . One can reduce the target weight, by observing that

Lemma 3. *Let $\Gamma : \mathbb{F}_{p^m} \rightarrow \mathbb{F}_p^m$ be the coordinate expansion map with respect to a fixed $\mathbb{F}_p$ -basis. Then for $x \in \mathbb{F}_{p^m}^*$ chosen uniformly at random, the expected Hamming weight of $\Gamma(x)$ is*

$$\mathbb{E}_{x \in \mathbb{F}_{p^m}^*} [\text{wt}(\Gamma(x))] = Fm, \quad \text{where} \quad F = \frac{p^m - p^{m-1}}{p^m - 1}.$$

Proof. For $1 \leq i \leq m$, define the indicator function

$$I_i(x) \triangleq \begin{cases} 1 & \text{if } i\text{-th coordinate of } \Gamma(x) \text{ is nonzero,} \\ 0 & \text{otherwise.} \end{cases}$$

Then the Hamming weight of $\Gamma(x)$ is $\text{wt}(\Gamma(x)) = \sum_{i=1}^m I_i(x)$, so

$$\mathbb{E}_x [\text{wt}(\Gamma(x))] = \frac{1}{p^m - 1} \sum_{i=1}^m \sum_{x \in \mathbb{F}_{p^m}^*} I_i(x).$$

For fixed i , the number of $x \in \mathbb{F}_{p^m}^*$ with nonzero i -th coordinate is $(p-1)p^{m-1}$, since we fix one coordinate to lie in $\mathbb{F}_p^*$ and let the remaining $m-1$ vary freely over $\mathbb{F}_p$. The result follows. $\square$

In Section 6.2, we provide novel BJMM-like algorithms over extension fields, which use information from the base field. In fact, one can equivalently state these algorithms directly over the base field. We play around with the weight of our entries in the base field to reduce base list sizes. However, this also leads to a reduced number of representations and ultimately converges to the cost of the classical q -ary BJMM.

Finally, in Sections 6.3 we use different ways to go to the base field, which is through the trace, and subfield subcodes.

We will focus on the case $\mathbb{F}_{2^m}$, as our main application (the first round submission of SDitH) considers this scenario, but want to point out that the algorithms work for any characteristic.

In several cases, we will only focus on small extensions such as $m = 2$, to lighten the notation.

6.1 Solve over Subfield

The driving question in this section is: can we translate the syndrome decoding problem over an extension field to a syndrome decoding problem in the base field? Since ISD algorithms are faster and more studied over $\mathbb{F}_2$, this might provide an improvement for ISD instances over $\mathbb{F}_{2^m}$.

To get to the base field, one approach is through the expansion map. Let $\mathbb{F}_{p^m}$ be an extension field of $\mathbb{F}_p$ and let $\Gamma = \{\xi_1, \xi_2, \dots, \xi_m\}$ be a basis of $\mathbb{F}_{p^m}$ over $\mathbb{F}_p$.

Let us denote by

$$\Gamma : \mathbb{F}_{p^m} \rightarrow \mathbb{F}_p^m$$

$$\sum_{i=1}^m x_i \xi_i \mapsto (x_1, \dots, x_m).$$

By abuse of notation we will denote by $\Gamma : \mathbb{F}_{p^m}^n \rightarrow \mathbb{F}_p^{mn}$ also its coordinate-wise extension. Clearly, Γ is an $\mathbb{F}_p$ -linear isomorphism.

Let $\mathcal{C} \subseteq \mathbb{F}_{p^m}^n$ of dimension k . The expanded code $\Gamma(\mathcal{C}) = \{\Gamma(\mathbf{c}) \mid \mathbf{c} \in \mathcal{C}\} \subseteq \mathbb{F}_p^{mn}$ has dimension mk , as $|\mathcal{C}| = |\Gamma(\mathcal{C})|$.

In order to keep the linear relation $\mathbf{e}\mathbf{H} = \mathbf{s}$, we expand $\mathbf{H}$ using

$$\Gamma(\mathbf{H}) = \begin{pmatrix} \Gamma(\xi_1 \mathbf{h}_1) & \cdots & \Gamma(\xi_m \mathbf{h}_1) & \cdots & \Gamma(\xi_1 \mathbf{h}_{n-k}) & \cdots & \Gamma(\xi_m \mathbf{h}_{n-k}) \end{pmatrix},$$

where $\mathbf{h}_i$ denote columns of $\mathbf{H}$. Since then we still have $\Gamma(\mathbf{e})\Gamma(\mathbf{H}) = \Gamma(\mathbf{s})$.

Thus, given the instance $\mathbf{H} \in \mathbb{F}_{p^m}^{(n-k) \times n}$, $\mathbf{s} \in \mathbb{F}_{p^m}^{n-k}$ and $\mathbf{e} \in \mathbb{F}_{p^m}^n$ of Hamming weight w , we construct an equivalent instance over $\mathbb{F}_p$ given by $\Gamma(\mathbf{H}) \in \mathbb{F}_p^{m(n-k) \times mn}$, $\Gamma(\mathbf{s}) \in \mathbb{F}_p^{m(n-k)}$, $\Gamma(\mathbf{e}) \in \mathbb{F}_p^{mn}$ having w many non-zero blocks of length m .

We propose two main approaches to solving the new syndrome decoding instance with information set decoding:

1. respect the block structure when choosing the information set.
2. forget the block structure and solve the new problem directly by picking entries from all blocks for the information set.

With the first approach, we are able to recover the exact runtime of the original algorithms, as the two formulations (over the extension field and over the base field) are equivalent. We then continue with tweaking this idea in Section 6.2. We also show that breaking the block structure even a little bit will give a worse algorithm.

6.1.1 Keep Blocks Together

For simplicity, we consider Prange’s algorithm. Here we are looking for an information set consisting of k many zero blocks out of n , knowing there are w many non-zero blocks. Thus, the average number of iteration stays unchanged as

$$\binom{n}{w}^{-1} \binom{n-k}{w}.$$

While there may be some practical improvements in implementation – such as performing Gaussian elimination on an $m(n-k) \times mn$ matrix over $\mathbb{F}_p$, which could be faster than on an $(n-k) \times n$ matrix over $\mathbb{F}_p^m$ – the overall runtime remains largely unaffected.

The same observation extends to the other ISD algorithms discussed in Section 3, i.e., T_{ISD} remains essentially unchanged.

6.1.2 Breaking Block-Structure

That leaves the possibility of forgetting the block structure, and tackling the syndrome decoding problem in the subfield directly. Here we are looking for a length nm vector $\Gamma(\mathbf{e})$ which satisfies $m(n-k)$ syndrome equations and has weight at most mw . Again for simplicity, let us just consider Prange’s algorithm. Here the average number of iterations changes to

$$\binom{m(n-k)}{wm}^{-1} \binom{nm}{wm}.$$

Up to subexponential factors, the runtime is thus

$$2^{(H(w/n) - H(w/(n-k)))nm} = T_{\text{Prange}}^m, \quad (5)$$

where T_{Prange} is the runtime of Prange’s algorithm on the original instance.

Thus, ignoring the $\mathbb{F}_{p^m}$ -linearity completely is not beneficial. One can do a little bit better by using the fact that a non-zero block has on average Fm many non-zero entries (with F defined in lemma (3), so with high probability, the vector $\Gamma(\mathbf{e})$ has weight close to Fmw , and not mw . But our experiments show that this is not enough to offset the enormous slowdown illustrated above.

Additionally, this approach also suffers from false-positives: there are several solutions to the instance over $\mathbb{F}_p$ which do not extend to the unique solution $\mathbf{e}$.

6.1.3 Mix of the Two

Despite the disheartening result of the previous section, we decided to try a mix of the two earlier ideas to see whether the optimum might lie somewhere in between. In the first idea all guesses of zero-positions fall within the same blocks, and in the second, we ignored the blocks completely. In this attempt, we introduced an extra parameter: N , which counts the number of blocks from which we guess that all positions are zero. The remaining zero-positions, we chose at random from all remaining positions in $\Gamma(\mathbf{e})$. See figure 2 for an illustration.



Figure 2: Different approaches to guessing zero-positions in $\Gamma(e)$

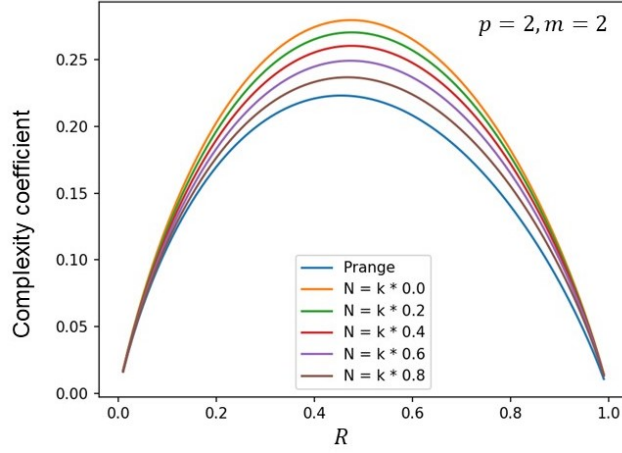


Figure 3: Runtime of the algorithm "mix of the two" with different values for N

The average number of iteration then becomes

$$\binom{n-w}{N}^{-1} \binom{(n-N)m - Fmw}{(k-N)m}^{-1} \binom{n}{N} \binom{(n-N)m}{(k-N)m},$$

where F is defined in lemma (3).

6.1.4 Results from Experiments and Conclusions

We ran experiments with the runtime above and optimized $N \in [0, k]$. We found that the optimal choice for N is k . In Figure 3 we illustrate the runtime for different values of N when $p = 2$ and $m = 2$. For larger characteristic p , the value of F increases and for larger extension degrees m , the slowdown described by Equation (5) is worse.

We thus conclude that breaking the block structure even a little bit is a bad idea.

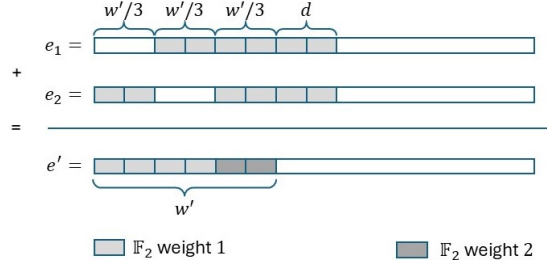


Figure 4: An illustration of final merge in the algorithm BJMM2_Build

6.2 Using Low-Weight Elements in Base Lists

In Section 5, we noted that for large q , modern ISD algorithms converge to Prange’s, largely because the base list size grows with q . This section explores a novel strategy to mitigate this issue: we design BJMM variants where the base lists are constrained to vectors whose entries are “small” field elements. The goal is to build up the final solution by combining these “small” elements into “bigger” ones.

To formalize this, we focus on extension fields of characteristic 2, i.e., $\mathbb{F}_{2^m}$. We fix a basis $\Gamma = (\xi_1, \dots, \xi_m)$ for $\mathbb{F}_{2^m}$ over $\mathbb{F}_2$ and define the “size” of an element by its $\mathbb{F}_2$ -weight.

Definition 4. Let $x \in \mathbb{F}_{2^m}$ have a basis expansion $x = \sum_{i=1}^m x_i \xi_i$ with $x_i \in \mathbb{F}_2$. The $\mathbb{F}_2$ -**weight** of x , denoted $\text{wt}_{\mathbb{F}_2}(x)$, is the Hamming weight of the coordinate vector $\Gamma(x) = (x_1, \dots, x_m)$.

The following subsections present two algorithms based on this principle.

6.2.1 A Hierarchical Weight Construction for BJMM

We first introduce a variant of the BJMM algorithm, which we call BJMM2_Build, that aims to reduce the size of the base lists. The core strategy is to restrict the vectors that comprise these lists. Specifically, the vectors in the base lists are constructed using only field elements of low $\mathbb{F}_2$ -weight. The final partial error vector, e' , is then built by combining these restricted vectors.

To illustrate this method, we focus on its application over $\mathbb{F}_4$. The algorithm constructs e' of weight w' as the sum of two vectors, e_1 and e_2 , each of weight w_1 . We assume that d of their entries cancel out. For a randomly chosen e' , we can assume with high probability that one-third of the non-zero entries of e' have an $\mathbb{F}_2$ -weight of 2, while the remaining two-thirds have an $\mathbb{F}_2$ -weight of 1. To produce this structure from e_1 and e_2 vectors containing only $\mathbb{F}_2$ -weight 1 elements, their weight must be

$$w_1 = 2w'/3 + d.$$

This construction is depicted in Figure 4. The vectors e_1 and e_2 are formed via a Stern-like merge of vectors from the base lists. These base lists contain

vectors $\mathbf{x} \in \mathbb{F}_4^{(k+\ell)/2}$ of weight $w_1/2$, where every entry has an $\mathbb{F}_2$ -weight of 1. This restriction leads to smaller base lists of size

$$B = \binom{(k+\ell)/2}{w_1/2} 2^{w_1/2}.$$

While the base list is smaller, this approach also affects the number of representations, ρ . This value is found by counting the pairs $(\mathbf{e}_1, \mathbf{e}_2)$ that sum to a fixed $\mathbf{e}'$ under the constraint that $\text{wt}(\mathbf{e}_1) = \text{wt}(\mathbf{e}_2) = w_1$ and all non-zero entries in $\mathbf{e}_1$ and $\mathbf{e}_2$ have $\mathbb{F}_2$ -weight 1. The total number of representations is given by

$$\rho = \binom{2w'/3}{w'/3} 2^{w'/3} \binom{k+\ell-w'}{d} 2^d,$$

where the factors arise as follows. The $\binom{2w'/3}{w'/3}$ term comes from allocating the $2w'/3$ positions of $\mathbb{F}_2$ -weight 1 in $\mathbf{e}'$ equally between $\mathbf{e}_1$ and $\mathbf{e}_2$. The $\binom{k+\ell-w'}{d}$ factor is for choosing the d cancellation positions among those where $\mathbf{e}'$ is zero. Finally, for each of these d cancellation positions and for each of the $w'/3$ positions that result in an $\mathbb{F}_2$ -weight 2 entry, there are two choices for the $\mathbb{F}_2$ -weight 1 element in $\mathbf{e}_1$ (which fixes the entry in $\mathbf{e}_2$), yielding the factors 2^d and $2^{w'/3}$.

The cost of the remaining parts of the algorithm, such as the intermediate lists L_1 and the final candidates C , follow the standard BJMM formulas, leading to a total round cost of

$$R_{\text{BJMM2_Build}} \in \tilde{O}(T_{\text{Gauss}} + B + L_1 + C),$$

and the expected number of rounds remains

$$\pi_{\text{BJMM2_Build}} \in \tilde{O} \left(\binom{k+\ell}{w'}^{-1} \binom{n-k-\ell}{w-w'}^{-1} \binom{n}{w} \right).$$

Results. We optimized the algorithm's parameters and plotted the complexity coefficient for half-distance decoding (see details of complexity coefficient calculations in Section 4). The results, shown in Figure 5 (left), confirm that BJMM2_Build does not improve on the existing BJMM algorithm with two levels.

We also extended the algorithm to $\mathbb{F}_{16}$ (see Appendix A), but the results shown in Figure 5 (right) were also negative. To quantify the comparison with BJMM2, we define a metric, $D(q)$, that measures the fractional improvement of our algorithm relative to the standard BJMM2. Formally, it is defined as follows:

$$D(q) \triangleq \frac{C_{\text{Prange}}^*(q) - C_{\text{BJMM2_Build}}^*(q)}{C_{\text{Prange}}^*(q) - C_{\text{BJMM2}}^*(q)}.$$

For $\mathbb{F}_4$ and $\mathbb{F}_{16}$, we found

$$D(4) = 0.075 \quad \text{and} \quad D(16) = 0.022,$$

which shows that for $\mathbb{F}_4$, the new algorithm achieves only 7.5% of the improvement that BJMM2 offers over Prange. For $\mathbb{F}_{16}$, this drops to just 2.2%. Given these discouraging results, we did not pursue larger field sizes.

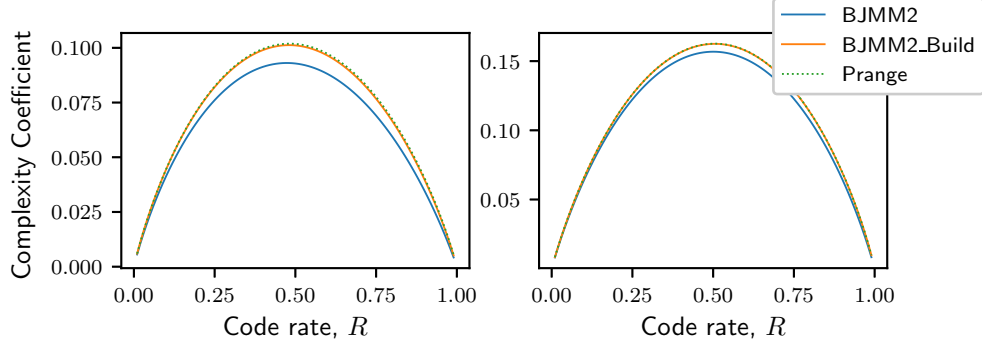


Figure 5: A comparison of the complexity coefficients for Prange, standard two-level BJMM (BJMM2), and the “BJMM2 Build” variant, plotted against the code rate for half-distance decoding. The plot on the left shows the performance over the finite field $\mathbb{F}_4$, while the plot on the right shows the results for $\mathbb{F}_{16}$.

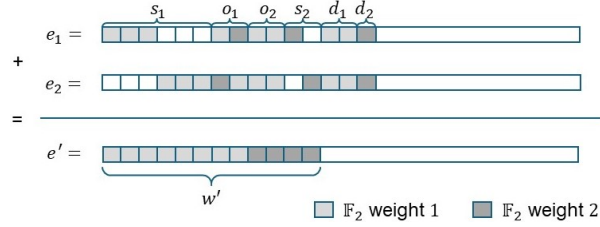


Figure 6: Introducing extra parameters to allow high $\mathbb{F}_2$ -weight entries in the base lists

6.2.2 A Mixed-Weight Base List Approach

The previous approach was highly restrictive. In this section, we instead investigate a middle ground between that and the original BJMM by allowing a mix of low and high $\mathbb{F}_2$ -weight elements in the base lists.

We introduce extra parameters illustrated in Figure 6 and defined for $i = 1, 2$ by

- s_i , the number of entries in e' of $\mathbb{F}_2$ -weight i that arise as a sum of a non-zero element of $\mathbb{F}_4$ and a zero.
- o_i , the number of entries in e' of $\mathbb{F}_2$ -weight i that arise as a sum of two non-zero elements of $\mathbb{F}_4$.
- d_i the number of entries in e_1 of $\mathbb{F}_2$ -weight i , which after adding e_2 cancel out.
- w_{1i} , the number of entries of $\mathbb{F}_2$ -weight i in the vectors in the base list,

i.e.,

$$\begin{aligned}w_{11} &= s_1/2 + o_1/2 + o_2 + d_1, \\w_{12} &= o_1/2 + s_2/2 + d_2.\end{aligned}$$

With this notation, the size of the base lists becomes

$$B = \binom{(k + \ell)/2}{w_{11}/2, w_{12}/2, \cdot} 2^{w_{11}/2}.$$

The number of representations ρ for this more complex structure is a combinatorial count of the ways to form $\mathbf{e}'$ given the new parameters:

$$\rho = \binom{2w'/3}{s_1/2, s_1/2, o_1/2, o_1/2} \binom{w'/3}{o_2, s_2/2, s_2/2} \binom{k + \ell - w'}{d_1, d_2, \cdot} 2^{d_1 + o_2}.$$

The remaining parts of the algorithm stay the same.

Results. We achieved the fastest algorithm when w_{11} was chosen equal to $2w_{12}$, i.e., when all field elements appeared the same number of times in the base list vectors. In this case we get exactly the runtime of original BJMM algorithm.

We took a quick look at the prime fields $\mathbb{F}_3$ and $\mathbb{F}_5$ as well. Here we tested the idea of having many 1's in the base lists, and then adding those together to form 2's in $\mathbb{F}_3$. And for $\mathbb{F}_5$ we tested base lists with 1's and -1's, and then adding them together to get 2's and 3's. We got similar results: the optimal choice is to have equally many of each field element in the base list vectors.

We conclude that at least with regards to BJMM variants with limitations on field elements in the base lists, the prime and extension fields behave the same.

6.2.3 Conclusion on Variants of BJMM with Limitations on Field Elements in Base Lists

While the proof of Section 5, suggests that the base list size is the main hindrance to BJMM being efficient for large fields and while the base list size does put a theoretic limitation on BJMM for $q \rightarrow \infty$, the base list size is only one factor influencing the final runtime. In fact, our attempts at decreasing this base list, in turn decreased the number of representations, which outweighed the gain from the smaller base lists.

While our experiments can never exhaust all possibilities, this provides evidence that the idea of excluding large $\mathbb{F}_2$ -weight elements from the base lists, will not improve the runtime.

6.3 Other Ways to Get to the Base Field

In the previous sections, we explored ISD algorithms that operate directly over the extension field $\mathbb{F}_{p^m}$ or translate the problem to the base field $\mathbb{F}_p$ via the

expansion map. The latter approach significantly worsened the problem parameters. In this section, we explore other methods for translating an ISD instance to its base field, which yield different parameter sets.

The main motivation for these approaches stems from two observations. First, as shown in Section 5, the performance of advanced ISD algorithms degrades as the field size q grows, making a translation to a smaller field potentially advantageous. Second, for the binary case ($\mathbb{F}_2$), even faster algorithms exist, such as the one by Both and May [10], which asymptotically outperforms BJMM. These potential benefits, however, must be weighed against the degradation of the code parameters that a translation to the base field incurs. We investigate this trade-off by exploring two main avenues: using the subfield subcode and using the trace map.

6.3.1 Using the Subfield Subcode

Let $\mathcal{C}^\perp \subseteq \mathbb{F}_{p^m}^n$ be the linear code generated by $\mathbf{H}$. If the subfield subcode $\mathcal{C}^\perp \cap \mathbb{F}_p^n$ is non-empty, we can use it to create syndrome equations over the subfield. In this section we investigate how many subfield equations we can expect to get, and how they might help us solve the syndrome decoding problem.

It turns out that the dimension of the subfield subcode is, with high probability, given by

$$r' = n - km.$$

for random codes. Hence the original problem $\mathbf{e}\mathbf{H} = \mathbf{s}$ is equivalent to a problem of the form

$$\begin{bmatrix} \mathbf{H}' \\ \mathbf{H}'' \end{bmatrix} \mathbf{e} = \tilde{\mathbf{s}},$$

where $\mathbf{H}' \in \mathbb{F}_p^{(n-km) \times n}$ and $\mathbf{H}'' \in \mathbb{F}_{p^m}^{k(m-1) \times n}$.

If we expand the entries of $\mathbf{e}$ and $\tilde{\mathbf{s}}$ over $\mathbb{F}_p$ as $e_j = \sum_{l=1}^m e_{jl}\xi_l$ and $\tilde{s}_i = \sum_{l=1}^m \tilde{s}_{il}\xi_l$, we can rewrite the first $n - mk$ syndrome equations as

$$\sum_{j=1}^n h'_{i,j} \sum_{l=1}^m e_{jl}\xi_l = \sum_{l=1}^m \tilde{s}_{il}\xi_l, \quad \text{for } i = 1, \dots, n - mk.$$

Since $h_{i,j} \in \mathbb{F}_p$ for all i, j , we can directly equate coefficients and find that

$$\sum_{j=1}^n h'_{i,j} e_{j1} = \tilde{s}_{i1}, \dots, \sum_{j=1}^n h'_{i,j} e_{jm} = \tilde{s}_{im}, \quad \text{for } i = 1, \dots, n - mk.$$

These last equations are interesting as all values are in the subfield. And we can group them into m groups, where each group only has the same n unknowns.

If we try to solve one of these groups directly via information set decoding, we get a problem with n unknowns, $n - km$ equations and in expectation Fmw non-zero unknowns over $\mathbb{F}_p$ (with F defined in Lemma 3). That is, leveraging the subfield subcode lowers the relative weight of the problem and allows us

work over $\mathbb{F}_p$, but it also means fewer equations. It is not obvious ahead of time which effect wins out – we will explore this balance numerically in Section 6.3.4.

The m subfield systems are not independent: their supports overlap much more than one would expect for random systems. One might hope to speed up decoding by solving them together, or by including the extra $k(m-1)$ equations over the larger field $\mathbb{F}_{p^m}$ with all mn unknowns. So far, though, we have not found a way to make use of these cross-constraints in practice.

6.3.2 Using the Trace Code

The trace function offers another path to the base field.

Definition 5 (Trace). Let $\mathbb{F}_{p^m}$ be a field extension of $\mathbb{F}_p$. The **trace** function $\text{Tr} : \mathbb{F}_{p^m} \rightarrow \mathbb{F}_p$ is defined by

$$\text{Tr}(x) = \sum_{i=0}^{m-1} x^{p^i}.$$

We also use Tr to denote the coordinate-wise application of the trace to a vector $\mathbf{v} \in \mathbb{F}_{p^m}^n$.

The trace is linear over the base field $\mathbb{F}_p$ but not over the extension field $\mathbb{F}_{p^m}$. Consequently, we cannot apply it directly to the syndrome equation $\mathbf{H}\mathbf{e} = \mathbf{s}$. Instead, we can reframe the SDP as an instance of the Codeword Finding Problem (defined below) and then transition to work in the trace code (the code found by taking the trace of all codewords of the code).

Definition 6 (Codeword Finding Problem). Given a code $\mathcal{C}$ and a weight w , the *Codeword Finding Problem (CFP)* asks to find a codeword $\mathbf{c} \in \mathcal{C}$ of Hamming weight w .

However, this approach is ultimately suboptimal. By Delsarte’s theorem, the dual of the trace code is equal to the subfield subcode of the dual code. This makes the resulting problem nearly identical to the one analyzed in the previous section. The key difference is that the initial conversion from SDP to CFP slightly degrades the problem’s effective parameters. Given that this method is a marginally worse variant of the subfield subcode approach, we do not pursue it further.

6.3.3 Use Trace to Get Some Unknowns in Subfield

The previous approaches attempted to translate the entire problem into the base field, which incurred a significant parameter cost. In this section, we explore a more modest aim: transforming the problem into a system where just *some* of the unknowns are in the base field.

We know that the trace is not linear over the extension field, and thus we cannot apply it directly to the system $\mathbf{H}\mathbf{e} = \mathbf{s}$. However, the trace is $\mathbb{F}_p$ -linear.

This suggests that if some entries in the matrix $\mathbf{H}$ lie in the base field $\mathbb{F}_p$, we can apply the trace fruitfully.

Our strategy, therefore, is to first achieve this condition. We perform Gaussian elimination to bring the parity-check matrix into a systematic form, which creates an identity submatrix whose entries (0s and 1s) are inherently in $\mathbb{F}_p$. With the system in this form, we can apply the trace operator Tr to obtain the following $n - k$ equations:

$$\begin{aligned} \text{Tr}(e_1) + \sum_{j=n-k+1}^n \sum_{i=0}^{s-1} h_{1,j}^{p^i} e_j^{p^i} &= \text{Tr}(s_1), \\ &\vdots \\ \text{Tr}(e_{n-k}) + \sum_{j=n-k+1}^n \sum_{i=0}^{s-1} h_{n-k,j}^{p^i} e_j^{p^i} &= \text{Tr}(s_{n-k}). \end{aligned}$$

The resulting system has a mixed structure:

- The first $n - k$ unknowns are now of the form $\text{Tr}(e_i)$, which are elements of the base field $\mathbb{F}_p$.
- The remaining k unknowns, e_j , remain in the extension field $\mathbb{F}_{p^m}$.

However, as the equations show, the system is no longer linear, and we cannot directly apply Information Set Decoding. To analyze this non-linear system's complexity, we can expand all variables over the base field. This yields a linear problem over $\mathbb{F}_p$ with $n - k$ equations and $n - k + mk$ unknowns. The expected Hamming weight of the solution vector in this expanded system is approximately $Fw_{\frac{n-k}{n}} + Fmw_{\frac{k}{n}}$, where F is defined in Lemma (3).

We plot the resulting runtime in the upcoming section and find that this approach is substantially less efficient than applying Prange's algorithm to the original problem. The significant increase in the number of unknowns without a corresponding increase in equations makes the problem much harder. While other methods for solving non-linear systems exist, the poor information-to-unknowns ratio in this linearized system suggests they are unlikely to offer a competitive advantage.

6.3.4 Results

We now compare the complexity of solving the SDP for the new instances over the base field derived in this section against solving the original problem. For instances over $\mathbb{F}_2$, we assume the use of the asymptotically fastest Both-May ISD algorithm with four levels [10], while for larger fields we use Prange's algorithm.

In Figure 7, we plot the resulting complexity coefficients for original problem over $\mathbb{F}_4$ (left) and $\mathbb{F}_{16}$ (right) and its various translations to the binary subfield. We only plot runtime when the parameters are below the Gilbert Varshamov

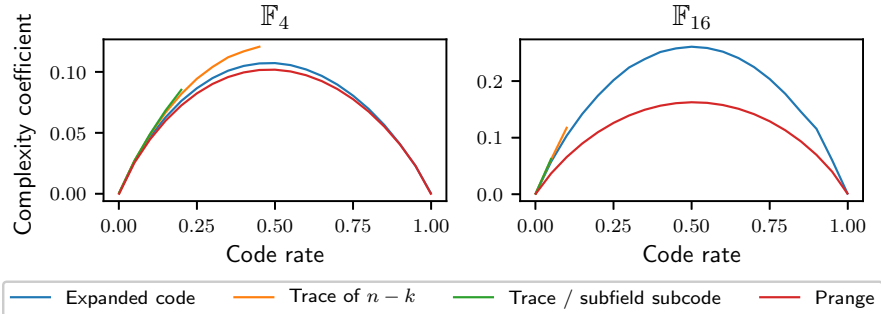


Figure 7: A comparison of complexity coefficients for various decoding strategies over $\mathbb{F}_4$ (left) and $\mathbb{F}_{16}$ (right). Prange’s algorithm serves as the baseline. The other curves show the complexity of solving the problem in the binary subfield via three methods: the Expanded code (Section 6.1), the Trace code / subfield subcode (Section 6.3.2/6.3.1), and the Trace of $n - k$ (Section 6.3.3). These binary instances are solved using the four-level Both-May algorithm [10]. Plots are shown only for rates where the error weight remains below the Gilbert-Varshamov distance to ensure a unique solution.

limit, and we can expect a unique solution. Once we go above this limit, the subfield instance will have rogue solutions, that do not extend to the correct solution in the extension field.

Our results conclusively show that translating the decoding problem to its base field is an inefficient strategy. Although this translation allows for the use of faster binary ISD algorithms, the benefits are negated by a significant degradation in code parameters, including increased code length and rate. Even the advantage of a lower relative error weight in the subfield is insufficient to offset this penalty. This finding reinforces our central conclusion: leveraging the base field structure through these methods does not appear to make decoding any easier.

7 Conclusion

This work investigated if the algebraic structure of extension fields can accelerate Information Set Decoding. We explored several methods, including base field expansion, trace codes, subfield subcodes and novel BJMM variants. However, contrary to our intuition stemming from related problems in the rank metric [21] and for rings [15], we found no evidence that these techniques make decoding asymptotically easier than over a prime field. For the approaches that translated the problem to the base field, a consistent trade-off emerged: while this often resulted in a problem with a lower relative error rate, the benefit was consistently outweighed by a costly increase in the code’s dimension and sometimes also its length. A key finding from our BJMM variants was a different critical trade-off:

gains from smaller base lists were nullified by a corresponding reduction in the number of representations.

Beyond these findings, this work offers two contributions of independent interest. First, as a new result, our asymptotic analysis shows that for small fields, a three-level BJMM offers a slight performance improvement over the standard two-level version. Second, we expanded on a proof by Meurer, showing that even for simultaneously growing parameters, the complexity of advanced ISD algorithms converges to that of Prange’s algorithm.

Despite these contributions, the central question of how to exploit field structure remains open, suggesting several promising avenues for future work. For instance, leveraging Frobenius automorphisms is an intriguing possibility, as this generates multiple, distinct systems of equations whose respective solutions all share the same support. The unsolved challenge is how to combine these correlated systems to accelerate the search for that support. Other promising avenues include finding novel ways to use equations from the subfield subcode or adapting paradigms like sieving and nearest-neighbor search specifically for the q -ary case. In summary, the distinction between prime and extension fields in code-based cryptography remains a rich and important area for future research.

Acknowledgements

We thank Andre Esser, Paolo Santini, Edoardo Persichetti and Christian Majenz for their insightful discussions and continuous advice throughout this research.

Funding

Freja Elbro was funded by Dencrypt in collaboration with the Danish Ministry of Defense Acquisition and Logistics Organization. Violetta Weger’s work was supported by the Technical University of Munich—Institute for Advanced Study.

Bibliography

- [1] Carlos Aguilar Melchor et al. *BIKE*. Fourth round finalist of the NIST Post-Quantum Cryptography Call, v. 4.2. Version 4.2. Sept. 2021. URL: <https://bikesuite.org>.
- [2] Carlos Aguilar Melchor et al. *HQC*. Selected Algorithm of the NIST Post-Quantum Cryptography Call. https://pqc-hqc.org/doc/hqc-specification_2021-06-06.pdf. June 2021.
- [3] Carlos Aguilar Melchor et al. *The Syndrome Decoding in the Head (SD-in-the-Head) Signature Scheme*. Submission to the NIST Post-Quantum Cryptography Call, v.1. May 2023. URL: <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-1/spec-files/SDitH-spec-web.pdf>.

- [4] Martin Albrecht et al. *Classic McEliece (merger of Classic McEliece and NTS-KEM)*. <https://classic.mceliece.org>. Fourth round finalist of the NIST post-quantum cryptography call. Oct. 2020.
- [5] Alexander Barg. “Some new NP-complete coding problems”. In: *Problemy Peredachi Informatsii* 30.3 (1994), pp. 23–28.
- [6] Alexander Barg and G David Forney. “Random codes: Minimum distances and error exponents”. In: *IEEE Transactions on Information Theory* 48.9 (2002), pp. 2568–2573.
- [7] Erich Barvínek, Ivan Daler, and Jan Francu. “Convergence of sequences of inverse functions”. In: *Archivum Mathematicum* 27.2 (1991), pp. 201–204.
- [8] Anja Becker et al. “Decoding Random Binary Linear Codes in $2^{n/20}$: How $1 + 1 = 0$ Improves Information Set Decoding”. In: *Eurocrypt 2012*. : Springer, 2012.
- [9] Elwyn Berlekamp, Robert McEliece, and Henk Van Tilborg. “On the inherent intractability of certain coding problems”. In: *IEEE Transactions on Information Theory* 24.3 (1978), pp. 384–386.
- [10] Leif Both and Alexander May. “Decoding linear codes with high error rate and its impact for LPN security”. In: *International Conference on Post-Quantum Cryptography*. Springer. 2018, pp. 25–46.
- [11] Kévin Carrier. “Near-collisions finding problem for decoding and recognition of error correcting codes,” PhD thesis. Sorbonne Université (France), June 2020. URL: <https://hal.science/tel-03370678v4>.
- [12] Kevin Carrier, Valerian Hatey, and Jean-Pierre Tillich. “Projective space stern decoding and application to SDitH”. In: *International Conference on Applied Cryptography and Network Security*. Springer. 2024, pp. 29–52.
- [13] Kevin Carrier et al. “Statistical decoding 2.0: Reducing decoding to LPN”. In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2022, pp. 477–507.
- [14] Kévin Carrier et al. “Reduction from sparse LPN to LPN, dual attack 3.0”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2024, pp. 286–315.
- [15] Giulia Cavicchioni, Alessio Meneghetti, and Giovanni Tognolini. “Information Set Decoding for Ring-Linear Code”. In: *Cryptology ePrint Archive* (2024).
- [16] Thomas Debris-Alazard and Jean-Pierre Tillich. “Statistical decoding”. In: *2017 IEEE International symposium on information theory (ISIT)*. IEEE. 2017, pp. 1798–1802.
- [17] Léo Ducas et al. “Asymptotics and improvements of sieving for codes”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2024, pp. 151–180.

- [18] Ilya Dumer. “On minimum distance decoding of linear codes”. In: *Proc. 5th Joint Soviet-Swedish Int. Workshop Inform. Theory*. Moscow, 1991, pp. 50–52.
- [19] Andre Esser and Emanuele Bellini. “Syndrome decoding estimator”. In: *IACR International Conference on Public-Key Cryptography*. Springer. 2022, pp. 112–141.
- [20] Andre Esser et al. “SoK: CryptographicEstimators - a Software Library for Cryptographic Hardness Estimation”. In: *AsiaCCS*. : {ACM}, 2024.
- [21] Philippe Gaborit, Olivier Ruatta, and Julien Schrek. “On the complexity of the rank syndrome decoding problem”. In: *IEEE Transactions on Information Theory* 62.2 (2015), pp. 1006–1019.
- [22] Edgar N Gilbert. “A comparison of signalling alphabets”. In: *The Bell system technical journal* 31.3 (1952), pp. 504–522.
- [23] Cheikh Thiécoumba Gueye, Jean Belo Klamti, and Shoichi Hirose. “Generalization of BJMM-ISD using May-Ozerov nearest neighbor algorithm over an arbitrary finite field”. In: *International Conference on Codes, Cryptology, and Information Security*. Springer. 2017, pp. 96–109.
- [24] Qian Guo, Thomas Johansson, and Vu Nguyen. “A new sieving-style information-set decoding algorithm”. In: *IEEE Transactions on Information Theory* (2024).
- [25] Shoichi Hirose. “May-Ozerov Algorithm for Nearest-Neighbor Problem over $\mathbb{F}_q$ and Its Application to Information Set Decoding”. In: *Innovative Security Solutions for Information Technology and Communications - 9th International Conference, SECITC 2016, Bucharest, Romania, June 9-10, 2016, Revised Selected Papers*. 2016, pp. 115–126. DOI: 10.1007/978-3-319-47238-6_8. URL: https://doi.org/10.1007/978-3-319-47238-6%5C_8.
- [26] Nick Howgrave-Graham and Antoine Joux. “New generic algorithms for hard knapsacks”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2010, pp. 235–256.
- [27] Alexander May, Alexander Meurer, and Enrico Thomae. “Decoding random linear codes in”. In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2011, pp. 107–124.
- [28] Alexander May and Ilya Ozerov. “On computing nearest neighbors with applications to decoding of binary linear codes”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2015, pp. 203–228.
- [29] Carlos Melchor et al. *The Syndrome Decoding in the Head (SD-in-the-Head) Signature Scheme*. Second round submission to the NIST post-quantum cryptography call. Feb. 2025. URL: <https://sdith.org/>.

- [30] Alexander Meurer. “A Coding-Theoretic Approach to Cryptanalysis”. PhD thesis. Ruhr University Bochum, Nov. 2012. URL: <http://www.cits.rub.de/imperia/md/content/diss.pdf>.
- [31] Raphael Overbeck. “Statistical decoding revisited”. In: *Information security and privacy : 11th Australasian conference, ACISP 2006*. Ed. by Reihaneh Safavi-Naini Lynn Batten. Vol. 4058. : Springer, 2006, pp. 283–294.
- [32] Christiane Peters. “Information-Set Decoding for Linear Codes over $\mathbf{F}_q$ ”. In: *PQCrypto 2010*. Vol. 6061. : Springer, 2010, pp. 81–94.
- [33] John Pierce. “Limit distribution of the minimum distance of random linear codes”. In: *IEEE Transactions on Information Theory* 13.4 (1967), pp. 595–599.
- [34] Eugene Prange. “The use of information sets in decoding cyclic codes”. In: *IRE Transactions on Information Theory* 8.5 (1962), pp. 5–9.
- [35] Jacques Stern. “A method for finding codewords of small weight”. In: *Coding Theory and Applications: 3rd International Colloquium Toulon, France, November 2–4, 1988 Proceedings 3*. Springer. 1989, pp. 106–113.
- [36] Rom Rubenovich Varshamov. “Estimate of the number of signals in error correcting codes”. In: *Doklady Akad. Nauk, SSSR* 117 (1957), pp. 739–741.

Appendix A Runtime for BJMM2_Build over $\mathbb{F}_{16}$

A simple generalization of the algorithm in Section 6.2.1 is an algorithm which builds elements of $\mathbb{F}_2$ -weight up to m by adding together elements of $\mathbb{F}_2$ -weight up to $m/2$. As m grows, this algorithm gets increasingly hard to describe, as it needs an increasing number of variables to describe how to add up to each $\mathbb{F}_2$ -weight. We describe here an algorithm for $\mathbb{F}_{16}$ as we consider it the most simple after $\mathbb{F}_4$, which we already described.

In $\mathbb{F}_{16}$, four elements have $\mathbb{F}_2$ -weight 1, six have $\mathbb{F}_2$ -weight 2, 4 have $\mathbb{F}_2$ -weight 3 and one has $\mathbb{F}_2$ -weight 4. For this algorithm, we assume that the entries of $\mathbf{e}'$ distribute among the different $\mathbb{F}_2$ -weights in those proportions, as this happens with overwhelming probability.

We will denote by

- o_i the number of entries of $\mathbf{e}'$ of $\mathbb{F}_2$ -weight i , which are formed as a sum of two non-zero elements of $\mathbb{F}_{16}$,
- s_i the number of entries of $\mathbf{e}'$ of $\mathbb{F}_2$ -weight i , which are formed as a sum of a zero and one non-zero element of $\mathbb{F}_{16}$,
- d_i the number of entries of $\mathbf{e}_1$ and $\mathbf{e}_2$ of $\mathbb{F}_2$ -weight i which cancel out,
- w_{1i} the total number of entries of $\mathbf{e}_1$ and $\mathbf{e}_2$ of $\mathbb{F}_2$ -weight i .

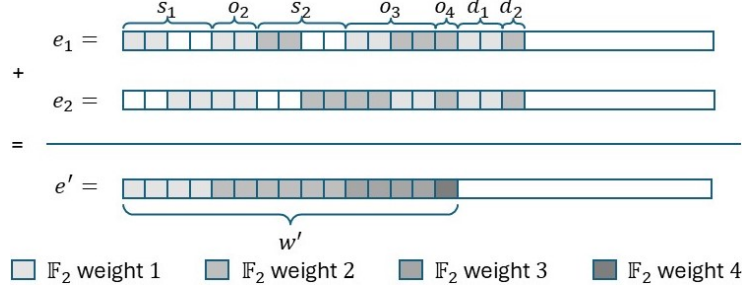


Figure 8: Final merge in an algorithm over $\mathbb{F}_{16}$ where elements of $\mathbb{F}_2$ -weight $\leq m$ are built by adding together elements of $\mathbb{F}_2$ -weight $\leq m/2$

With the above notation

$$w_{11} = s_1/2 + o_2 + o_3/2 + d_1$$

and

$$w_{12} = s_2/2 + o_3/2 + o_4 + d_2.$$

See figure 8. If we only allow elements of $\mathbb{F}_2$ -weight up to $m/2$ in the base lists, this brings the base list size to

$$B = \binom{(k+l)/2}{w_{11}/2, w_{12}/2, \cdot} 4^{w_{11}/2} 6^{w_{12}/2}$$

and the number of representations to

$$\rho = \binom{s_1}{s_1/2} \binom{2w'/5}{o_2, s_2/2, s_2/2} 2^{o_2} \binom{o_3}{o_3/2} 3^{o_3} \binom{4}{2}^{o_4} \binom{k+l-w'}{d_1, d_2, \cdot}.$$

The remaining runtime calculations stay the same.